```
print(filtered_df['terms'].apply(type).value_counts())
```

```
terms
<class 'list'>    7944
Name: count, dtype: int64
```

```
!python -m spacy download en_core_web_sm
```

```
lels/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
.2.8 MB 99.9 MB/s eta 0:00:00

›_web_sm')

ıeed to restart Python in
ın do this by selecting the
```

```
import spacy
from spacy.matcher import Matcher

# 2. Download and load the en_core_web_sm spaCy model.
nlp = spacy.load('en_core_web_sm')
print("spaCy model 'en_core_web_sm' loaded successfully.")

# 3. Define a Python function, extract_noun_phrases
def extract_noun_phrases(doc):
    return [chunk.text for chunk in doc.noun_chunks]

# 4. Define a Python function, extract_named_entities
def extract_named_entities(doc):
    return [ent.text for ent in doc.ents]

# 5. Import Matcher from spacy.matcher is already done.
# 6. Initialize a Matcher object and add patterns to identify technical terms.
mMatcher = Matcher(nlp.vocab)

# Example patterns for technical terms:
# Pattern 1: Adjective + Noun (e.g., 'natural language')
pattern1 = [{"POS": "ADJ"}, {"POS": "NOUN"}]
# Pattern 2: Noun + Noun (e.g., 'machine learning')
pattern2 = [{"POS": "NOUN"}, {"POS": "NOUN"}]
# Pattern 3: Proper Noun + Noun (e.g., 'Google Cloud')
pattern3 = [{"POS": "PROPN"}, {"POS": "NOUN"}]
# Pattern 4: Noun (singular or plural) ending with 'system' or 'model'
pattern4 = [{"POS": {"IN": ["NOUN", "PROPN"]}, "LENGTH": {"<": 15}, "TEXT": {"REGEX": ".*
# Pattern 5: Two or more capitalized words (potential acronyms or proper nouns acting as
pattern5 = [{"IS_ALPHA": True, "IS_UPPER": True, "OP": "+", "LENGTH": {">=": 2}}]


mMatcher.add("TECHNICAL_TERM", [pattern1, pattern2, pattern3, pattern4, pattern5])

# 7. Define a Python function, extract_technical_terms
def extract_technical_terms(doc):
```

```
    matches = mMatcher(doc)
    terms = []
    for match_id, start, end in matches:
        span = doc[start:end]  # The matched span of tokens
        terms.append(span.text)
    return terms

print("Functions 'extract_noun_phrases', 'extract_named_entities', and 'extract_technical
```

```
_web_sm' loaded successfully.
oun_phrases', 'extract_named_entities', and 'extract_technical_terms' defined successfully.
```

```
print("Processing 'summaries' column to extract noun phrases, named entities, and technic

# 1. Process the 'summaries' column using nlp.pipe()
filtered_df['doc'] = list(nlp.pipe(filtered_df['summaries']))

# 2. Apply extract_noun_phrases function
filtered_df['noun_phrases'] = filtered_df['doc'].apply(extract_noun_phrases)

# 3. Apply extract_named_entities function
filtered_df['named_entities'] = filtered_df['doc'].apply(extract_named_entities)

# 4. Apply extract_technical_terms function
filtered_df['technical_terms'] = filtered_df['doc'].apply(extract_technical_terms)

# 5. Optionally drop the temporary 'doc' column
filtered_df = filtered_df.drop(columns=['doc'])

print("Extraction complete. New columns 'noun_phrases', 'named_entities', and 'technical_
print(filtered_df[['summaries', 'noun_phrases', 'named_entities', 'technical_terms']].hea
```

```
Processing 'summaries' column to extract noun phrases, named entities, and technical terms
/tmp/ipython-input-648660964.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_gu
  filtered_df['doc'] = list(nlp.pipe(filtered_df['summaries']))
/tmp/ipython-input-648660964.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_gu
  filtered_df['noun_phrases'] = filtered_df['doc'].apply(extract_noun_phrases)
/tmp/ipython-input-648660964.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_gu
  filtered_df['named_entities'] = filtered_df['doc'].apply(extract_named_entities)
Extraction complete. New columns 'noun_phrases', 'named_entities', and 'technical_terms' a
                                            summaries  \
1   The recent advancements in artificial intellig...
2   In this paper, we proposed a novel mutual cons...
7   To mitigate the radiologist's workload, comput...
26  Image segmentation is often ambiguous at the l...
34  Deep reinforcement learning augments the reinf...
```

```
                                        noun_phrases  \
1   [The recent advancements, artificial intellige...
2   [this paper, we, a novel mutual consistency ne...
7   [the radiologist's workload, computer-aided di...
26  [Image segmentation, the level, individual ima...
34  [Deep reinforcement learning, the reinforcemen...

                                       named_entities  \
1   [today, AI, AI, AI, five, European, AI, FUTURE...
2         [two, First, Second, one, five, three, two]
7                             [two, Tile, AutoEncoder]
26  [Segmenter, first, Vision Transformer, ViT, AD...
34                                  [seven, 2D, 3D]

                                      technical_terms
1   [recent advancements, artificial intelligence,...
2   [mutual consistency, consistency network, MC, ...
7   [medical images, interest segmentation, exciti...
26  [Image segmentation, individual image, context...
34  [Deep reinforcement, reinforcement learning, p...
/tmp/ipython-input-648660964.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_gu
  filtered_df['technical_terms'] = filtered_df['doc'].apply(extract_technical_terms)
```

```
print("Processing 'summaries' column to extract noun phrases, named entities, and technic

# Ensure filtered_df is a standalone copy to avoid SettingWithCopyWarning
filtered_df = filtered_df.copy()

# 1. Process the 'summaries' column using nlp.pipe()
filtered_df['doc'] = list(nlp.pipe(filtered_df['summaries']))

# 2. Apply extract_noun_phrases function
filtered_df['noun_phrases'] = filtered_df['doc'].apply(extract_noun_phrases)

# 3. Apply extract_named_entities function
filtered_df['named_entities'] = filtered_df['doc'].apply(extract_named_entities)

# 4. Apply extract_technical_terms function
filtered_df['technical_terms'] = filtered_df['doc'].apply(extract_technical_terms)

# 5. Optionally drop the temporary 'doc' column
filtered_df = filtered_df.drop(columns=['doc'])

print("Extraction complete. New columns 'noun_phrases', 'named_entities', and 'technical_
print(filtered_df[['summaries', 'noun_phrases', 'named_entities', 'technical_terms']].hea
```

```
Processing 'summaries' column to extract noun phrases, named entities, and technical terms
Extraction complete. New columns 'noun_phrases', 'named_entities', and 'technical_terms' a
                                           summaries  \
1   The recent advancements in artificial intellig...
2   In this paper, we proposed a novel mutual cons...
7   To mitigate the radiologist's workload, comput...
26  Image segmentation is often ambiguous at the l...
```

```
34  Deep reinforcement learning augments the reinf...

                                          noun_phrases  \
1   [The recent advancements, artificial intellige...
2   [this paper, we, a novel mutual consistency ne...
7   [the radiologist's workload, computer-aided di...
26  [Image segmentation, the level, individual ima...
34  [Deep reinforcement learning, the reinforcemen...

                                         named_entities  \
1   [today, AI, AI, AI, five, European, AI, FUTURE...
2         [two, First, Second, one, five, three, two]
7                             [two, Tile, AutoEncoder]
26  [Segmenter, first, Vision Transformer, ViT, AD...
34                                  [seven, 2D, 3D]

                                        technical_terms
1   [recent advancements, artificial intelligence,...
2   [mutual consistency, consistency network, MC, ...
7   [medical images, interest segmentation, exciti...
26  [Image segmentation, individual image, context...
34  [Deep reinforcement, reinforcement learning, p...
```

```
from collections import Counter

print("Aggregating noun phrases and calculating frequencies...")

# Flatten the list of lists of noun phrases
all_noun_phrases = [phrase for sublist in filtered_df['noun_phrases'] for phrase in subli

# Calculate the frequency of each noun phrase
noun_phrase_counts = Counter(all_noun_phrases)

# Get the top 20 most frequent noun phrases
top_noun_phrases = noun_phrase_counts.most_common(20)

print("Top 20 most frequent noun phrases:")
for phrase, count in top_noun_phrases:
    print(f"- {phrase}: {count}")
```

```
Aggregating noun phrases and calculating frequencies...
Top 20 most frequent noun phrases:
- we: 12852
- We: 9372
- that: 7136
- which: 5961
- it: 3051
- this paper: 2433
- the-art: 2186
- they: 1122
- this work: 1065
- RL: 1005
- -: 996
- reinforcement learning: 877
- them: 776
- This: 758
- It: 724
- our method: 701
- this: 685
```

```
- training: 666
- This paper: 650
- data: 648
```

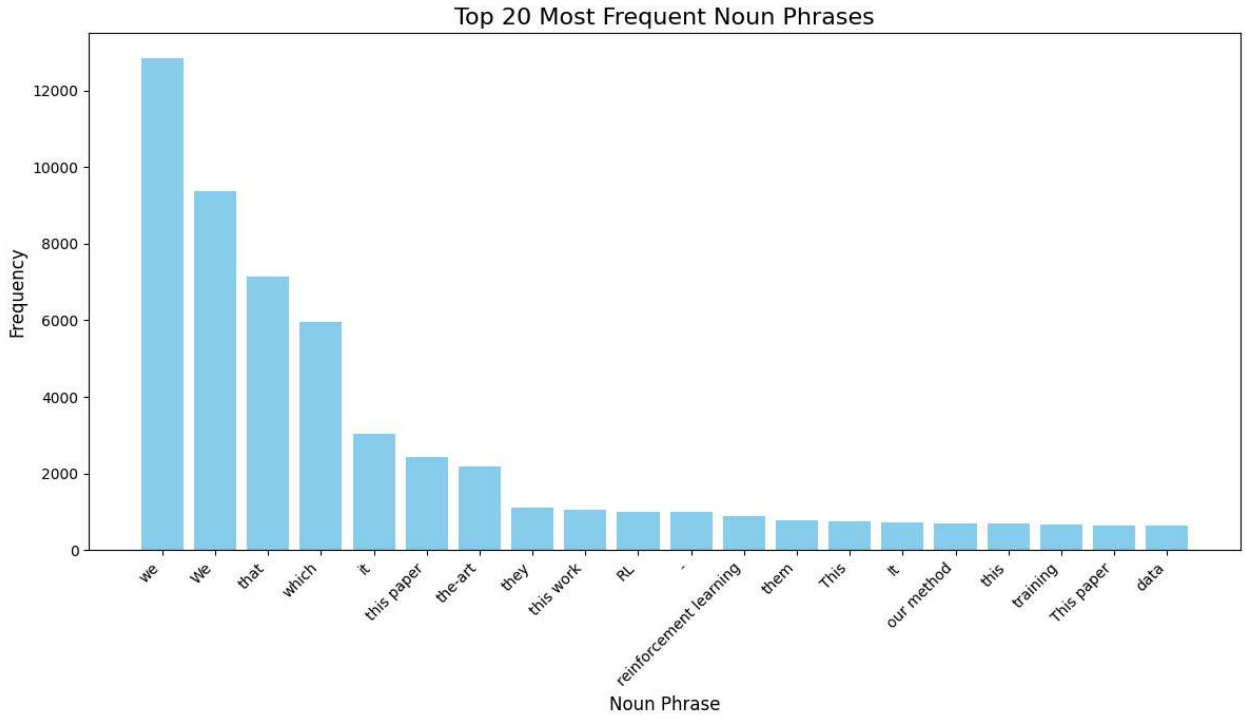
```
from collections import Counter

print("Aggregating named entities by type and calculating frequencies...")

# Ensure filtered_df is a standalone copy to avoid SettingWithCopyWarning
filtered_df = filtered_df.copy()

# 1. Create a temporary 'doc' column by processing 'summaries'
filtered_df['doc'] = list(nlp.pipe(filtered_df['summaries']))

# 2. Initialize a list to store (entity_text, entity_type) tuples
all_named_entities_with_types = []

# 3. Iterate through the 'doc' column and extract entities with their types
for doc_obj in filtered_df['doc']:
    for ent in doc_obj.ents:
        all_named_entities_with_types.append((ent.text, ent.label_))

# 4. Use collections.Counter to calculate the frequency of each (entity_text, entity_type)
named_entity_type_counts = Counter(all_named_entities_with_types)

# 5. Get the top 20 most frequent named entities with their types
top_named_entities_with_types = named_entity_type_counts.most_common(20)

# 6. Print the top 20 named entities
print("\nTop 20 most frequent named entities with their types:")
for (entity, ent_type), count in top_named_entities_with_types:
    print(f"- {entity} (Type: {ent_type}): {count}")

# 7. Drop the temporary 'doc' column from filtered_df
filtered_df = filtered_df.drop(columns=['doc'])

print("\nNamed entity analysis complete.")
```

```
Aggregating named entities by type and calculating frequencies...

Top 20 most frequent named entities with their types:
- two (Type: CARDINAL): 2342
- RL (Type: PERSON): 1526
- first (Type: ORDINAL): 1389
- RL (Type: ORG): 1085
- three (Type: CARDINAL): 905
- one (Type: CARDINAL): 763
- 2 (Type: CARDINAL): 523
- 1 (Type: CARDINAL): 512
- CNN (Type: ORG): 496
- GAN (Type: ORG): 427
- GNN (Type: ORG): 360
- DRL (Type: ORG): 312
- 3 (Type: CARDINAL): 280
- four (Type: CARDINAL): 279
- AI (Type: GPE): 277
- First (Type: ORDINAL): 274
- 2D (Type: CARDINAL): 271
```

```
- second (Type: ORDINAL): 249
- Transformer (Type: ORG): 249
- recent years (Type: DATE): 244

Named entity analysis complete.
```

```
import matplotlib.pyplot as plt

print("Generating bar chart for top noun phrases...")

# Extract noun phrases and their counts
phrases = [phrase for phrase, count in top_noun_phrases]
counts = [count for phrase, count in top_noun_phrases]

# Create the bar chart
plt.figure(figsize=(12, 7)) # Adjust figure size for better readability
plt.bar(phrases, counts, color='skyblue')

# Add title and labels
plt.title('Top 20 Most Frequent Noun Phrases', fontsize=16)
plt.xlabel('Noun Phrase', fontsize=12)
plt.ylabel('Frequency', fontsize=12)

# Rotate x-axis labels for better readability
plt.xticks(rotation=45, ha='right')

# Adjust plot layout
plt.tight_layout()

# Display the plot
plt.show()

print("Bar chart displayed successfully.")
```

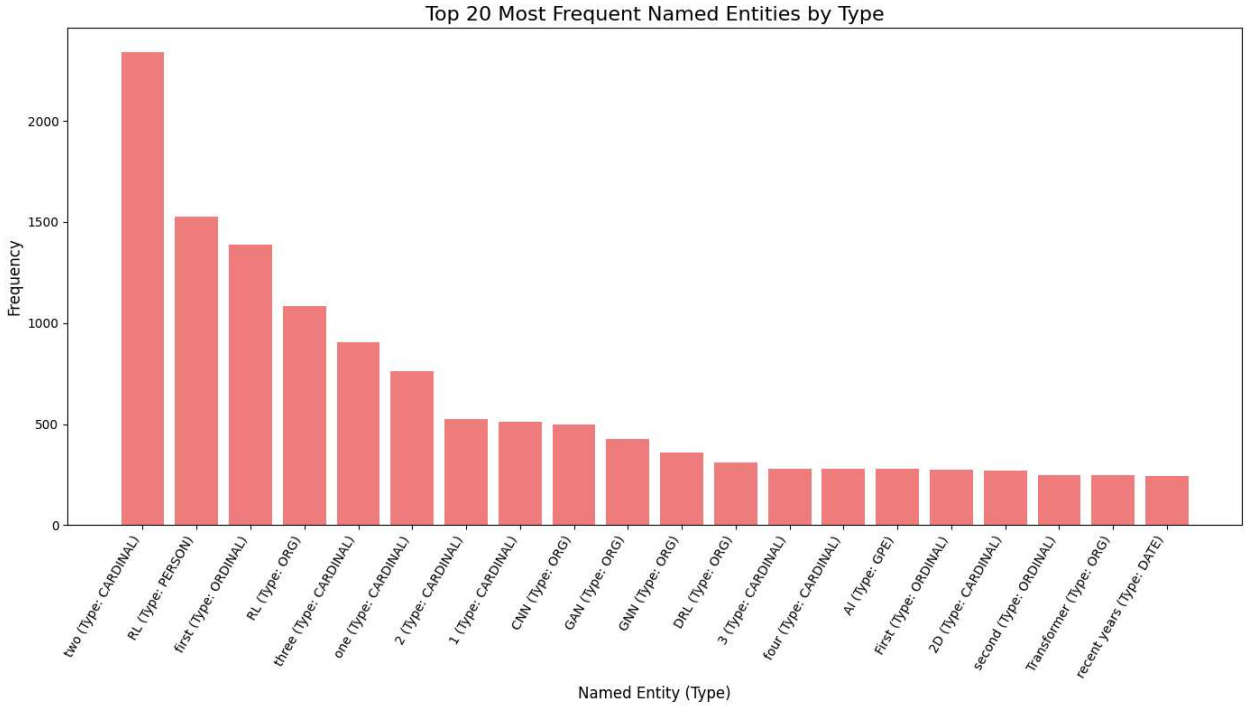
```
Generating bar chart for top noun phrases...
```




```
Bar chart displayed successfully.
```

```
import matplotlib.pyplot as plt

print("Generating bar chart for top named entities by type...")

# Extract named entities, their types, and counts
entities_labels = [f"{entity} (Type: {ent_type})" for (entity, ent_type), count in top_na
entities_counts = [count for (entity, ent_type), count in top_named_entities_with_types]

# Create the bar chart
plt.figure(figsize=(14, 8)) # Adjust figure size for better readability
plt.bar(entities_labels, entities_counts, color='lightcoral')

# Add title and labels
plt.title('Top 20 Most Frequent Named Entities by Type', fontsize=16)
plt.xlabel('Named Entity (Type)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)

# Rotate x-axis labels for better readability
plt.xticks(rotation=60, ha='right')

# Adjust plot layout
plt.tight_layout()

# Display the plot
plt.show()

print("Bar chart for named entities displayed successfully.")
```

```
Generating bar chart for top named entities by type...
```




```
Bar chart for named entities displayed successfully.
```

```
print("Selecting and processing a representative abstract for tokenization demonstration.

# 1. Select a representative abstract (e.g., the first one) from the 'summaries' column c
representative_abstract = filtered_df['summaries'].iloc[0]

# 2. Process this abstract using the nlp() object to create a spaCy Doc object.
doc = nlp(representative_abstract)

print("Abstract selected and processed into a spaCy Doc object.")
print(f"Selected Abstract: {representative_abstract[:200]}...")
```

```
Selecting and processing a representative abstract for tokenization demonstration...
Abstract selected and processed into a spaCy Doc object.
Selected Abstract: The recent advancements in artificial intelligence (AI) combined with t
extensive amount of data generated by today's clinical systems, has led to the
development of imaging AI solutions across the ...
```

```
print("Demonstrating tokenization with spaCy...")

# 3. Iterate through each token in the Doc object.
# 4. For each token, print its text, lemma, part-of-speech (POS) tag, and dependency relat
print("--------------------------------------------------------------------------------
print(f"{'Text':<20} {'Lemma':<20} {'POS':<10} {'Dependency':<15} {'Is_alpha':<10} {'Is_st
print("--------------------------------------------------------------------------------
for token in doc:
    print(f"{token.text:<20} {token.lemma_:<20} {token.pos_:<10} {token.dep_:<15} {str(tok
print("--------------------------------------------------------------------------------

print("Tokenization demonstration complete.")
```

```
and                 and                 CCONJ      cc              True       True
(                   (                   PUNCT      punct           False      False
vi                  vi                  PROPN      nmod            True       False
)                   )                   PUNCT      punct           False      False
Explainability      explainability      NOUN       conj            True       False
.                   .                   PUNCT      punct           False      False
In                  in                  ADP        prep            True       True
a                   a                   DET        det             True       True
step                step                VERB       nmod            True       False
-                   -                   PUNCT      punct           False      False
by                  by                  ADP        prep            True       True
-                   -                   PUNCT      punct           False      False
step                step                NOUN       pobj            True       False
approach            approach            NOUN       pobj            True       False
,                   ,                   PUNCT      punct           False      False
these               these               DET        det             True       True
guidelines          guideline           NOUN       nsubjpass       True       False
are                 be                  AUX        auxpass         True       True


                    SPACE      dep             False      False
further             far                 ADV        advmod          True       True
translated          translate           VERB       ROOT            True       False
into                into                ADP        prep            True       True
a                   a                   DET        det             True       True
framework           framework           NOUN       pobj            True       False
of                  of                  ADP        prep            True       True
concrete            concrete            ADJ        amod            True       False
recommendations     recommendation      NOUN       pobj            True       False
for                 for                 ADP        prep            True       True
specifying          specify             VERB       pcomp           True       False
,                   ,                   PUNCT      punct           False      False


                    SPACE      dep             False      False
developing          develop             VERB       conj            True       False
,                   ,                   PUNCT      punct           False      False
evaluating          evaluate            VERB       conj            True       False
,                   ,                   PUNCT      punct           False      False
and                 and                 CCONJ      cc              True       True
deploying           deploy              VERB       conj            True       False
technically         technically         ADV        advmod          True       False
,                   ,                   PUNCT      punct           False      False
clinically          clinically          ADV        advmod          True       False
and                 and                 CCONJ      cc              True       True
ethically           ethically           ADV        advmod          True       False


                    SPACE      dep             False      False
trustworthy         trustworthy         ADJ        amod            True       False
AI                  AI                  PROPN      compound        True       False
```